

Ejercicio 2

Considera el siguiente escenario asumiendo el rol de ingeniero de datos:

El área operativa del departamento de TI tiene tres aplicaciones. Para cada aplicación su fuente de datos es de tipo transaccional y las tres funcionan 24/7. Llamemosles F1, F2 y F3 respectivamente, las cuales poseen las siguientes características:

- F1 es un CRM propietario. Contiene información sobre los clientes e.g. demográficos y datos de contacto.
- F2 tiene un RDBMS SQL Server. Almacena las transacciones de los clientes sobre la mitad de los productos que ofrece la compañía.
- F3 tiene un RDBMS Postgresql. Almacena las transacciones de los clientes sobre el resto de los productos.

Ten en cuenta los siguientes supuestos:

- Las tres fuentes de datos F1, F2 y F3 fueron diseñadas de manera independiente tanto a nivel de software como de modelo de datos.
- Como las fuentes de datos son operativas diariamente hay datos nuevos.

Tu objetivo es diseñar una arquitectura que consolide la información contenida en las fuentes de datos de F1, F2 y F3 que cumpla con dos propósitos: en primer lugar que habilite a un grupo de usuarios del área operativa a la extracción de consultas por medio de SQL y de manera secundaria que permita al equipo de ciencia de datos la aplicación de algoritmos de detección de patrones como clustering o búsquedas en grafos.

Tu diseño puede utilizar herramientas de almacenamiento y procesamiento diferentes para cada propósito si lo consideras necesario.

A. De cada fuente de datos se tienen identificados que campos requiere el área operativa. ¿Para cumplir con los dos objetivos que subconjunto de cada fuente de datos extraerías?

Extraería un subconjunto pensado para los dos consumos:

1. **Consulta operativa SQL:** datos limpios, actuales y entendibles.
2. **Ciencia de datos:** datos históricos, relacionales y con suficiente granularidad para patrones, clustering.

No extraería solo los campos que pidió el área operativa, porque para ciencia de datos normalmente se necesitan campos adicionales para construir comportamiento, relaciones, historial y trazabilidad. Extraería de F1 los datos maestros del cliente, como identificadores, datos demográficos, contacto, ubicación y estatus. De F2 y F3 extraería las transacciones: cliente, producto, monto, fecha, canal, sucursal y estado de la transacción.

Además, agregaría campos técnicos como sistema origen, llave origen, tipo de operación CDC y timestamp de ingesta. Con esto cubro las consultas SQL operativas y también dejo suficiente información para ciencia de datos, clustering .

B. ¿Qué posibles retos implica la extracción de cada una de las fuentes de datos por separado y qué herramientas utilizas?

F1 - CRM propietario

Retos:

- Puede no tener acceso directo a base de datos.
- Puede depender de APIs, archivos o exportaciones.
- Puede tener límites de consulta o paginación.
- Los datos pueden venir en JSON, CSV o formatos no estándar.
- Puede ser difícil detectar cambios incrementales.

Herramientas que usaría:

- **Airflow** para programar la extracción.
- **Python + Polars/Pandas** para consumir API o archivos.
- **MinIO** para guardar la data cruda en Bronze.
- **Iceberg** para almacenar la versión estructurada.

F2 - SQL Server

Retos:

- No impactar una base transaccional 24/7.
- Manejar inserts, updates y deletes.
- Extraer datos de forma incremental.
- Problemas de performance si se hacen cargas completas.
- Tipos de datos propios de SQL Server.

Herramientas que usaría:

- **CDC** para capturar cambios.
- **Airflow** para orquestar.
- **PySpark** si el volumen es alto.
- **MinIO** para almacenar los datos.
- **Iceberg** para manejar merges, updates y deletes.

F3 - PostgreSQL

Retos:

- Configurar CDC o logical replication.
- Manejar cambios constantes porque opera 24/7.
- No afectar el rendimiento de PostgreSQL.
- Homologar sus transacciones con las de SQL Server.
- Manejar diferencias de tipos de datos y esquemas.

Herramientas que usaría:

- **CDC** usando logical replication.
- **Airflow** para orquestación.
- **PySpark o Polars** para transformar.
- **MinIO** para landing.
- **Iceberg** para tablas consolidadas.
- **Trino** para consultar después.

C. ¿Qué posibles retos implica la independencia en el modelo de datos de las tres fuentes y cómo los resolverías?

El principal reto es que las tres fuentes pueden manejar identificadores, nombres de campos, tipos de datos, catálogos y reglas de negocio diferentes. Para resolverlo mantendría primero una capa Bronze con los datos crudos, y después construiría una capa Silver donde se aplique homologación, limpieza, deduplicación y estandarización.

Ejemplos:

Reto	Ejemplo	Cómo lo resolvería
IDs diferentes	El cliente tiene customer_id en F1, client_id en F2 y person_id en F3	Crear una tabla maestra de clientes o customer_master
Nombres de columnas distintos	phone, telefono, mobile_number	Homologar nombres en la capa Silver
Tipos de datos diferentes	Fecha como string en una fuente y timestamp en otra	Estandarizar tipos de datos durante la transformación
Catálogos diferentes	Estado A, ACTIVO, ACTIVE	Crear tablas de homologación/catálogos
Datos duplicados	El mismo cliente existe en varias fuentes	Aplicar reglas de deduplicación y matching

D. ¿Aparte de un proceso batch en la hora de menor uso, cómo podrías mitigar el impacto de tu pipeline sobre las fuentes originales ?

1. Usar CDC en lugar de cargas completas

Para SQL Server y PostgreSQL usaría **CDC** para leer solo cambios (UPDATES, INSERTS, DELETES)

2. Leer desde réplicas, no desde la base primaria

3. Extracción incremental

E. ¿Cuáles etapas considerarías en tu proceso de transformación de datos y qué uso les darías?

Consideraría tres etapas. Primero Bronze, donde guardo los datos crudos desde las fuentes con metadata de ingesta. Después Silver, donde limpio, valido, homologó esquemas, estandarizo tipos de datos, deduplico y unifico clientes y transacciones.

Finalmente Gold, donde publico tablas listas para consumo: modelos operativos consultables por SQL en Trino y datasets analíticos para ciencia de datos, como features para clustering o estructuras de nodos y aristas para grafos.

F. ¿Qué herramientas utilizas para las etapas de transformación?

BRONCE

Objetivo: recibir y guardar datos crudos.

Herramientas:

- Airflow
- Python
- MinIO
- Iceberg

SILVER

Objetivo: limpiar, homologar y transformar.

Herramientas:

- PySpark
- Polars
- Iceberg

- Hive Metastore

Gold

Objetivo: publicar datos listos para operación y ciencia de datos.

Herramientas:

- PySpark
- Polars
- Trino
- Iceberg
- MinIO

G. ¿Qué storage usarías para cada propósito y por qué ?

Usaría MinIO como storage principal tipo S3 para el data lake. En Bronze guardaría los datos crudos separados por fuente. Para los datos limpios y consolidados usaría tablas Iceberg sobre MinIO, porque permiten transacciones, evolución de esquema, particiones y consultas eficientes.

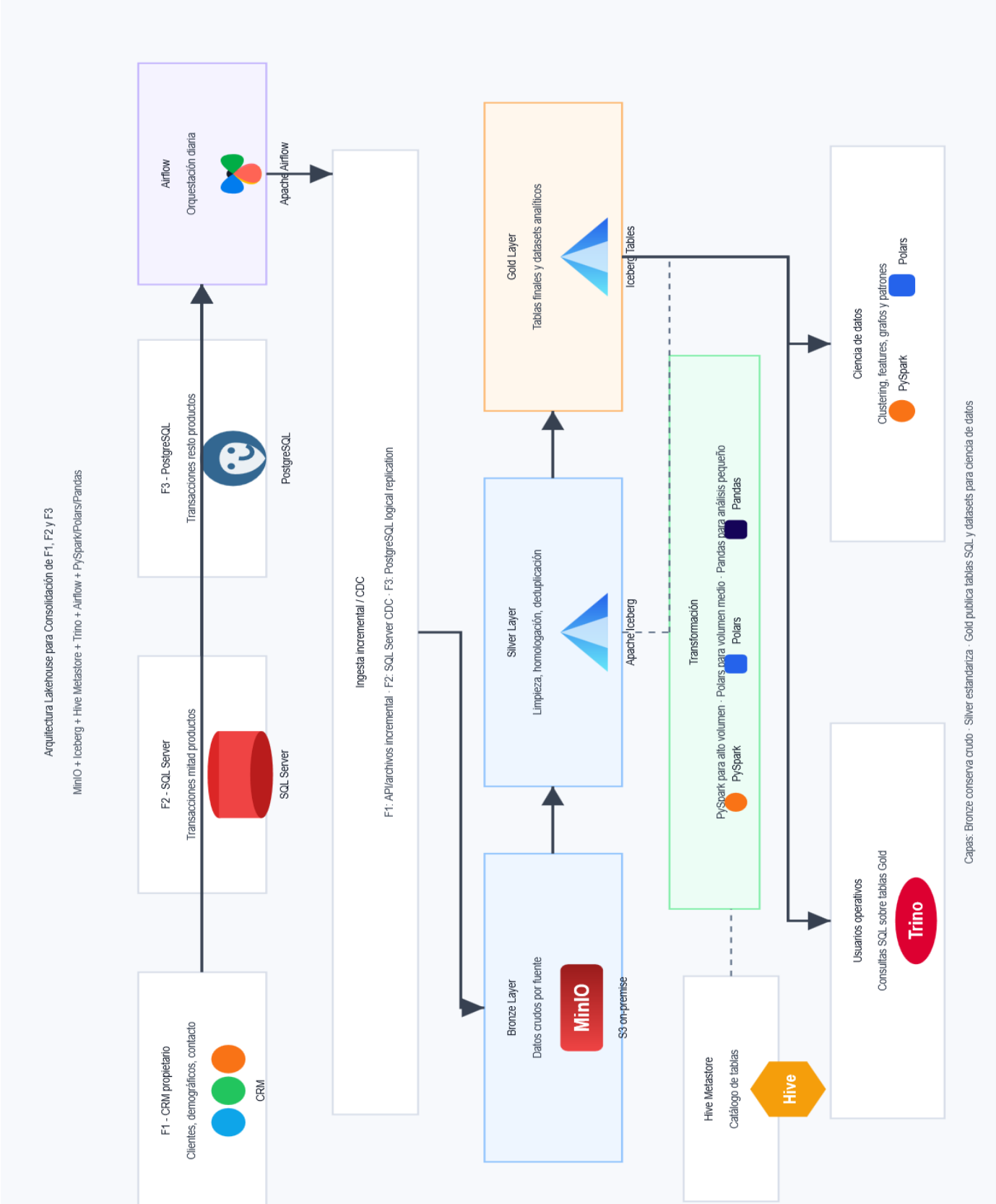
Para los usuarios operativos expondría las tablas Gold con Trino. Para ciencia de datos usaría también Iceberg/Parquet sobre MinIO, pero con tablas de features, nodos y aristas preparadas para PySpark, Polars o Pandas. Como catálogo usaría Hive Metastore.

H. Recuerda que al menos a diario tendrás que llevar data nueva a tu etapa de transformación final, ¿Como orquestarías tu pipeline y con qué herramienta?

Orquestaría el pipeline con Airflow mediante un DAG diario e incremental. El DAG extraería cambios desde F1, F2 y F3, cargaría la información en Bronze sobre MinIO, ejecutaría transformaciones hacia Silver y Gold con PySpark o Polars, y escribiría las tablas finales en Iceberg.

Además, controlaría reintentos, dependencias, logs, alertas, concurrencia y checkpoints como `last_execution_timestamp` o `cdc_offset` para evitar reprocesos o duplicados.

I. Proporciona un diagrama de tu propuesta de arquitectura.



II. Seguridad (manteniendo tu rol de ingeniero de datos).

A. ¿Cómo mantendrías la seguridad de tu flujo de datos end-to-end? Es decir disminuir riesgos de posibles fugas o intrusiones no deseadas al entorno de ejecución que estás construyendo.

1. Seguridad en la extracción

- Usar conexiones cifradas: **TLS/SSL** hacia F1, SQL Server y PostgreSQL.
- Usar usuarios técnicos con **mínimos permisos**.
- No consultar tablas completas si no es necesario.
- Usar **CDC** o réplicas de lectura para evitar tocar directamente la base productiva.
- Guardar credenciales en un **secret manager**, no en código ni DAGs.

2. Seguridad en Airflow

- Separar ambientes: dev, test y prod.
- Limitar quién puede editar DAGs.
- Auditar ejecuciones, errores y cambios.

3. Seguridad en almacenamiento MinIO

- Políticas por rol.
- Separar rutas por capa

4. Seguridad en Iceberg / Hive Metastore / Trino

- Controlar permisos por esquema y tabla.
- Exponer a usuarios solo la capa **Gold**.
- Evitar acceso directo a Bronze si contiene datos sensibles.
- Aplicar masking o anonimización en campos sensibles.

5. Protección de datos sensibles

- Identificar PII: nombre, teléfono, email, dirección, documento.
- Enmascarar o tokenizar datos sensibles.
- Cifrar columnas críticas si aplica.
- Minimizar datos: extraer solo lo necesario.
- Separar datasets identificables de datasets analíticos.

III. Gobernanza de datos

A. ¿Cómo llevarías control de la metadata y sus cambios al igual que los procesos de tu pipeline y cómo almacenamos estos datos?

Usaría Hive Metastore como catálogo central de metadata para registrar tablas Iceberg, esquemas, columnas y particiones. Para controlar cambios de datos y esquemas aprovecharía las capacidades de versionado, snapshots y schema evolution de Iceberg.

La metadata operativa del pipeline la almacenaría en PostgreSQL mediante Airflow y tablas de auditoría, registrando ejecuciones, errores, batch_id, volumen procesado y tiempos de ejecución. Además, mantendría versionado del código y configuraciones en Git para asegurar trazabilidad y control de cambios end-to-end.

1. Metadata de tablas y datasets

Hive Metastore como catálogo principal.

2. Control de cambios de datos y esquema

Capacidades de Iceberg.

3. Metadata del pipeline

Airflow + tablas de auditoría.

4. Versionamiento del pipeline

Todo el código lo controlaría en Git.